Azure Service Bus – Duplicate Message Detection

1. What is Duplicate Detection?

Duplicate Detection is a feature of Azure Service Bus that prevents the same message from being added to a queue multiple times.

Azure Service Bus identifies duplicate messages based on the **Message ID (`MessageId`)**.

Important Point

Azure Service Bus **does not compare the message body/content** to determine whether two messages are duplicates.

It checks the **Message ID**.

```
Same MessageId
       ↓
Azure Service Bus considers it a duplicate
       ↓
Duplicate message is not added
```

---

2. How Does Duplicate Detection Work?

Suppose we send two messages:

```
Message 1
MessageId = 101
Body = "Hello"

Message 2
MessageId = 101
Body = "Hello"
```

Both messages have the same `MessageId`.

Azure Service Bus identifies the second message as a duplicate and does not add it to the queue.

Result

```
Queue

Message 1 → Added
Message 2 → Duplicate → Not added
```

Therefore, only **one message** is available in the queue.

---

3. Message Content Is Not Used for Duplicate Detection

This is a very important interview point.

Consider:

```
Message 1
MessageId = 101
Body = "Hello"

Message 2
MessageId = 102
Body = "Hello"
```

The message contents are exactly the same.

However, the `MessageId` values are different.

Therefore, Azure Service Bus treats them as **two different messages**.

Result

```
Message 1 → Added
Message 2 → Added
```

So the queue contains **2 messages**.

Remember

> Same content does NOT mean duplicate.

Same `MessageId` means duplicate.

---

4. Example 1 – Same Content but Different Message IDs

Suppose we create two messages with the same content:

```
var message1 = new ServiceBusMessage("Message sent once");

var message2 = new ServiceBusMessage("Message sent once");
```

We have not explicitly specified a `MessageId`.

The two messages have the same body:

```
Message sent once
```

But they are separate messages.

Since they have different automatically generated message IDs, Azure Service Bus treats them as different messages.

Result

```
Message 1 → Added
Message 2 → Added
```

The queue contains:

```
2 messages
```

---

5. Example 2 – Same Content and Same Message ID

Now consider:

```
var message3 = new ServiceBusMessage("Message sent only once");

var message4 = new ServiceBusMessage("Message sent only once");

message3.MessageId = "Message-001";
message4.MessageId = "Message-001";
```

Here:

```
Message 3
MessageId = Message-001

Message 4
MessageId = Message-001
```

Both messages have the same `MessageId`.

When we send both messages:

```
await sender.SendMessageAsync(message3);
await sender.SendMessageAsync(message4);
```

Azure Service Bus identifies the second message as a duplicate.

Result

```
Message 3 → Added
Message 4 → Duplicate → Not added
```

Therefore, only **one message** is added to the queue.

---

6. Visual Example

Case 1 – Same Content, Different Message IDs

```
Message 1
----------------
MessageId = 101
Body = "Hello"
```

```
Message 2
----------------
MessageId = 102
Body = "Hello"
```

Although the content is the same:

```
MessageId 101 ≠ MessageId 102
```

Therefore:

```
Queue
 ├── Message 1
 └── Message 2
```

Total messages = 2

Case 2 – Same Message ID

```
Message 1
----------------
MessageId = 101
Body = "Hello"


Message 2
----------------
MessageId = 101
Body = "Hello"
```

Here:

```
MessageId 101 = MessageId 101
```

Therefore:

```
Queue
 └── Message 1
```

```
Message 2 → Duplicate → Rejected
```

Total messages = 1

---

7. Duplicate Detection Must Be Enabled

Duplicate detection is a **queue/topic configuration feature**.

It needs to be enabled on the Service Bus entity.

Conceptually:

```
Queue
   ↓
Duplicate Detection = Enabled
   ↓
Service Bus checks MessageId
   ↓
Duplicate Message?
   ├── Yes → Ignore duplicate
   └── No  → Add message
```

When duplicate detection is enabled, Service Bus maintains information about recently seen message IDs within the configured duplicate detection history window.

---

8. Duplicate Detection Window

Duplicate detection is not an unlimited historical check.

Azure Service Bus uses a **duplicate detection history window**.

For example:

```
MessageId = Order-1001
       ↓
Message received
       ↓
Service Bus remembers MessageId
       ↓
```

```
Same MessageId received within detection window
        ↓
Duplicate
```

After the relevant detection window has passed, the same `MessageId` may be accepted again.

Important

Duplicate detection is therefore mainly useful for preventing **accidental duplicate sends/retries within the configured detection window**.

---

9. Why Do We Need Duplicate Detection?

Duplicate detection is useful in distributed systems because a sender may not know whether a previous send succeeded.

For example:

```
Application
    ↓
Send Order Message
    ↓
Network Problem
    ↓
Application does not know whether message was accepted
    ↓
Application retries
```

Without duplicate detection:

```
Order Message
Order Message
```

The same order could appear twice.

With duplicate detection:

```
First Send
    ↓
MessageId = ORDER-1001
    ↓
```

```
Accepted

Retry
    ↓
MessageId = ORDER-1001
    ↓
Duplicate
    ↓
Ignored
```

This helps prevent duplicate processing caused by retries.

---

10. Real-Time Example – Order Processing

Imagine an e-commerce application.

An order is created:

```
OrderId = ORD1001
```

The application sends:

```
MessageId = ORD1001
Body = "Process Order ORD1001"
```

Due to a network issue, the application retries the operation using the same `MessageId`:

```
MessageId = ORD1001
Body = "Process Order ORD1001"
```

Azure Service Bus can recognize the second message as a duplicate when duplicate detection is enabled.

Result

```
First message  → Accepted
Second message → Duplicate → Ignored
```

This helps avoid processing the same order multiple times.

---

11. Important Difference: Message ID vs Message Body

Property	Used for Duplicate Detection?
MessageId	Yes
Message Body	No
Message Content	No
Other message properties	Not the primary duplicate key

Example

```
Message A
MessageId = 100
Body = "ABC"

Message B
MessageId = 101
Body = "ABC"
```

Same body, different IDs:

Result: 2 messages

```
Message A
MessageId = 100
Body = "ABC"

Message B
MessageId = 100
Body = "XYZ"
```

Same ID, different body:

Result: Duplicate detection can treat the second message as a duplicate.

This demonstrates why you should not assume that duplicate detection compares message contents.

---

12. Key Interview Question

Q: How does Azure Service Bus identify duplicate messages?

Answer:

Azure Service Bus identifies duplicate messages primarily using the **Message ID (`MessageId`)**. When duplicate detection is enabled, if another message with the same `MessageId` is sent within the configured duplicate detection history window, Service Bus considers it a duplicate and does not add the duplicate message to the queue.

---

13. Interview Question

Q: Does Azure Service Bus compare the message body to detect duplicates?

Answer:

No.

Azure Service Bus does not compare the message body/content to determine whether messages are duplicates. Duplicate detection is based on the **MessageId** within the configured duplicate detection history window.

---

14. Interview Question

Q: What happens if two messages have the same content but different MessageIds?

Answer:

They are treated as **two different messages** because duplicate detection does not compare the message body.

Example:

```
Message 1 → MessageId = 101, Body = "Hello"
Message 2 → MessageId = 102, Body = "Hello"
```

Result:

```
2 messages
```

---

15. Interview Question

Q: What happens if two messages have the same MessageId?

Answer:

If duplicate detection is enabled and the messages fall within the configured duplicate detection history window, Service Bus identifies the second message as a duplicate and does not add it to the queue.

Example:

```
Message 1 → MessageId = 101
Message 2 → MessageId = 101
```

Result:

```
Message 1 → Added
Message 2 → Duplicate → Ignored
```

---

16. Example from the Video

Initially, the Azure Service Bus queue contained:

```
Messages = 0
```

The application created two messages with the same content but did not explicitly specify the same `MessageId`.

Result

```
Message 1 → Added
Message 2 → Added
```

Queue:

```
Messages = 2
```

Then the application created two more messages with:

```
Same Content
+
Same MessageId
```

Result

```
Message 3 → Added
Message 4 → Duplicate → Ignored
```

Therefore, the final queue contained:

```
2 + 1 = 3 messages
```

This demonstrates that **MessageId, not message content, is used for duplicate detection**.

---

17. Best Practice

When implementing duplicate detection, use a **meaningful and stable MessageId**.

For example:

```
OrderId
PaymentId
TransactionId
InvoiceId
```

Example:

```
var message = new ServiceBusMessage("Process Order");
message.MessageId = "ORDER-1001";
```

If the same order needs to be retried, use the same logical `MessageId`:

```
MessageId = ORDER-1001
```

This allows Service Bus duplicate detection to recognize the retry as a duplicate when it falls within the detection window.

---

18. Quick Revision

```
Azure Service Bus Duplicate Detection
                ↓
       Checks MessageId
                ↓
       Same MessageId?
          /
        YES           NO
         ↓             ↓
    Duplicate      New Message
        ↓             ↓
    Ignored         Accepted
```

Remember These 5 Points

1. Duplicate detection prevents duplicate messages from being accepted.
2. It uses **MessageId**, not the message body.
3. Same content + different MessageId = **two messages**.
4. Same MessageId within the detection window = **duplicate**.
5. Duplicate detection must be enabled/configured on the Service Bus entity.

One-Line Interview Answer

> **Azure Service Bus duplicate detection uses the MessageId to identify duplicate messages; it does not compare the message body, and the check applies within the configured duplicate detection history window.**